

セッション管理ルール設計書

1. セッション管理の目的

- 同一アカウントでの手動操作と自動更新の競合を回避
- ログアウトを防止
- セッション状態の維持と復元
- 並列処理時の安全性確保

2. セッション管理の基本方針

2.1 アカウント単位のセッション管理

- 各アカウントは独立したセッションを持つ
- アカウントごとにブラウザコンテキストを分離
- セッション情報はデータベースに保存

2.2 セッション状態の監視

- 定期的なセッション状態チェック（5秒間隔）
- Session Tokenの変化検知
- DOM操作の監視
- ログイン状態の確認

3. 手動操作検知ルール

3.1 検知方法

Session Token変化検知

```
# 前回のTokenと現在のTokenを比較
if current_token != previous_token:
    # 自動操作以外のToken変化を検知
    if not is_automated_operation():
        set_manual_operation_flag()
```

DOM操作監視

```
# 自動操作以外のDOM変更を検知
def detect_manual_operation():
    # MutationObserverでDOM変更を監視
    # 自動操作フラグが立っていない変更を検知
    if dom_changed and not automation_flag:
        set_manual_operation_flag()
```

3.2 検知ウィンドウ

- 検知ウィンドウ: 30秒
- この期間内に手動操作が検知された場合、フラグを設定
- フラグ設定後、該当アカウントの処理をスキップ

3.3 フラグ管理

```
class ManualOperationFlag:
    def __init__(self):
        self.flag = False
        self.detected_at = None
        self.account_id = None

    def set(self, account_id):
        self.flag = True
        self.detected_at = datetime.now()
        self.account_id = account_id
        self.log_manual_operation()

    def clear(self, account_id):
        if self.account_id == account_id:
            self.flag = False
            self.detected_at = None
            self.account_id = None
```

4. セッション競合回避ルール

4.1 処理前チェック

```
def process_account(account_id):
    # セッション競合チェック
    if check_manual_operation(account_id):
        log_skip(account_id, reason="MANUAL_OPERATION_DETECTED")
        return SKIP

    # セッション状態確認
    if not check_session_valid(account_id):
        restore_session(account_id)

    # 処理実行
    execute_task(account_id)
```

4.2 処理中チェック

```
def execute_task_with_monitoring(account_id):
    # バックグラウンドでセッション監視
```

```
monitor = start_session_monitor(account_id)

try:
    # メイン処理
    result = execute_task(account_id)
    return result
except SessionConflictError:
    # セッション競合検知
    log_skip(account_id, reason="SESSION_CONFLICT")
    return SKIP
finally:
    # 監視停止
    stop_session_monitor(monitor)
```

4.3 スキップルール

- 手動操作が検知されたアカウントのみスキップ
- 他アカウントへの影響はゼロ
- スキップ時は必ずログを記録
- 次回実行時に再度チェック

5. セッション復元ルール

5.1 復元が必要なケース

1. ログアウト検知

- セッション期限切れ
- 手動ログアウト
- サーバー側のセッション無効化

2. アプリ停止

- 「赤い×」検知
- ブラウザクラッシュ

3. ネットワークエラー

- 接続タイムアウト
- 接続切断

5.2 復元手順

```
def restore_session(account_id):
    # 1. 既存セッション終了
    close_browser_context(account_id)

    # 2. 新しいセッション開始
    context = create_browser_context(account_id)
```

```
# 3. URL自動判別
login_url = get_login_url(account_id)

# 4. ログイン処理
login(account_id, login_url)

# 5. セッション情報保存
save_session_info(account_id, context)

# 6. 処理再開
resume_task(account_id)
```

6. 並列処理ルール

6.1 並列処理の制限

- 最大同時処理数: 10アカウント
- リソース使用量を考慮した動的調整
- アカウントごとに独立したキュー

6.2 キュー管理

```
class AccountQueue:
    def __init__(self, max_concurrent=10):
        self.max_concurrent = max_concurrent
        self.active_tasks = {}
        self.pending_tasks = Queue()

    def add_task(self, account_id, task):
        if len(self.active_tasks) < self.max_concurrent:
            self.execute_task(account_id, task)
        else:
            self.pending_tasks.put((account_id, task))

    def execute_task(self, account_id, task):
        # セッション競合チェック
        if check_manual_operation(account_id):
            self.skip_task(account_id)
            return

        # タスク実行
        future = executor.submit(task, account_id)
        self.active_tasks[account_id] = future

        # 完了時の処理
        future.add_done_callback(
            lambda f: self.on_task_complete(account_id)
        )
```

7. セッション情報の保存

7.1 保存する情報

```
class SessionInfo:
    account_id: str
    login_url: str # doors1 or doors2
    session_token: str
    cookies: dict
    last_activity: datetime
    browser_context_id: str
    status: str # ACTIVE, INACTIVE, CONFLICT
```

7.2 保存先

- データベース: **sessions**テーブル
- メモリ: 実行時のセッション情報
- ファイル: バックアップ用 (暗号化)

8. セッション有効性チェック

8.1 チェック項目

1. ログイン状態

- ログインページにリダイレクトされていないか
- セッションCookieの有効性

2. セッションToken

- Tokenの存在確認
- Tokenの有効期限確認

3. ページ状態

- 正常なページが表示されているか
- エラーページが表示されていないか

8.2 チェック頻度

- 処理前: 必ずチェック
- 処理中: 5秒間隔でチェック
- 処理後: 結果確認時にチェック

9. ログ記録ルール

9.1 記録する情報

- セッション開始時刻
- セッション終了時刻

- 手動操作検知時刻
- セッション復元時刻
- セッション競合検知時刻

9.2 ログフォーマット

```
{  
  "timestamp": "2024-12-20T10:30:45.123Z",  
  "account_id": "account_001",  
  "event_type": "SESSION_CONFLICT_DETECTED",  
  "details": {  
    "reason": "MANUAL_OPERATION",  
    "action": "SKIP",  
    "session_id": "session_12345"  
  }  
}
```

10. セッション管理のベストプラクティス

1. 定期的なクリーンアップ

- 無効なセッションの削除
- 古いセッション情報のアーカイブ

2. リソース管理

- ブラウザコンテキストの適切な終了
- メモリリークの防止

3. エラーハンドリング

- セッション関連エラーの適切な処理
- 自動復旧機能の実装

4. 監視とアラート

- セッション競合の頻度監視
- 異常なパターンの検知